Linux / Shell / Vim Notes (Deduplicated)

Prepared from user-provided markdown • 2026-03-29

在 **Mac 终端 / zsh / bash** 里，常见的是：

- **Ctrl + U**：删除光标前面这一整行
- **Ctrl + K**：删除光标后面的内容
- **Ctrl + A**：跳到行首
- **Ctrl + E**：跳到行尾
- **Ctrl + L**：清屏
- **Ctrl + C**：中断当前命令

所以你刚才如果命令输乱了，按 **Ctrl + U** 很有用。

好，这一块我直接给你一份 **“能面试 + 能日常用”的精华版 Cheat Sheet**（简洁但高频）

Part 1 — zsh 常用快捷键（高频）

编辑命令行（必会）

```
Ctrl + A    # 跳到行首
Ctrl + E    # 跳到行尾
Ctrl + U    # 删除光标前全部
Ctrl + K    # 删除光标后全部
Ctrl + W    # 删除前一个单词
Ctrl + L    # 清屏（等价 clear）
```

历史命令（超级常用）

```
↑ / ↓        # 上一条 / 下一条命令
Ctrl + R     # 搜索历史命令（面试高频）
!!           # 执行上一条命令
!git         # 执行最近的 git 开头命令
```

面试加分：

> “Ctrl+R is very useful to search command history.”

终端控制

```
Ctrl + C    # 终止程序
Ctrl + Z    # 挂起程序（后台）
```

```
fg          # 恢复
jobs        # 查看任务
```

Part 2 — zsh 自动补全（提升效率）

Tab 自动补全

```
git ch + Tab → git checkout
cd Do + Tab → cd Downloads
```

智能路径补全

```
cd ~/Dow + Tab
```

连按两次 Tab

```
Tab Tab
```

✅ 高亮（错误提示）

```
brew install zsh-syntax-highlighting
```

错误命令会变红

Part 4 — Linux / Shell 高频命令（面试必问）

文件操作

```
ls -l         # 列表
ls -a         # 包含隐藏文件
cd            # 切目录
pwd           # 当前路径
```

文件查看

```
cat file      # 全部输出
less file     # 分页查看（推荐）
head -n 10    # 前10行
tail -n 10    # 后10行
```

搜索（超高频）

```
grep "error" file.log
grep -r "hello" .
```

面试：

> “grep is used to search text patterns.”

文件操作

```
cp a b        # 复制
mv a b        # 移动/重命名
rm file      # 删除
rm -rf dir    # 删除目录（危险）
```

系统监控

```
ps aux        # 查看进程
top           # 实时监控
df -h         # 磁盘
du -sh *      # 文件大小
```

网络

```
ping google.com
curl http://example.com
wget http://example.com/file
```

管道（面试重点）

```
cat file | grep "error"
ps aux | grep python
```

面试一句：

> “Pipe passes output of one command as input to another.”

Part 5 — 高频组合（实战）

查日志错误

```
grep "ERROR" app.log | tail -20
```

找大文件

```
du -sh * | sort -h
```

找进程

```
ps aux | grep spark
```

面试必背总结

- Terminal = UI
- zsh/bash = shell
- grep = search
- pipe `|` = 数据流
- Ctrl+R = 查历史

如果你想更强（下一步）

我可以帮你

做一个 **“Linux + Shell 面试 30题（带答案）”** 或整理 **Spark + Linux + Git 一页速查表（面试神器）**

你现在这一步已经是 **SDE / Data Engineer 必备基础了 **

很好，这个是 **grep 面试高频点**

一句话

`-r` = recursive（递归搜索）

解释

```
grep -r "hello"
```

意思是：

➡️ **在当前目录 + 所有子目录中搜索 "hello"**

举个例子

目录结构：

```
project/
├── file1.txt
├── dir1/
│   ├── file2.txt
│   └── file3.txt
```

执行：

```
grep -r "hello" .
```

会搜索：

- file1.txt ✅
- dir1/file2.txt ✅
- dir1/file3.txt ✅

❌ 如果没有 -r

```
grep "hello" file1.txt
```

只能搜一个文件，不能遍历目录

常见组合（面试必问）

递归 + 显示行号

```
grep -rn "hello" .
```

- `-r`：递归
- `-n`：显示行号

忽略大小写

```
grep -ri "hello" .
```

`-i` = ignore case

只显示文件名

```
grep -rl "hello" .
```

`-l` = list file names

面试标准回答

`-r` means recursive search, which allows grep to search through all subdirectories.

总结

```
-r = recursive（递归查找目录）
```

如果你想更强

我可以帮你

总结 **grep / awk / sed 面试三件套（超高频）** 或出一套 **Linux 面试真题（带答案）**

你现在这个问题已经是 **Linux 面试基础核心点了 **

Linux / Shell 文件查看与操作速查

对，这里核心是在练 Linux / shell 里怎么看文件、怎么看目录、怎么统计行数。

---

1. 什么是 directory?

directory = 目录 / 文件夹

例如：

- `01_RDD` 是 directory
- `input` 是 directory

目录里可以包含：

- 文件（file）
- 子目录（subdirectory）

总结：

- file = 文件
- directory = 文件夹 / 目录

---

2. 为什么 `less` 只能看文件?

`less` 是用来 **查看文件内容** 的命令。

```
less a.txt   # ✅ 正确（文件）
less input   # ❌ 错误（目录）
```

原因：

- 文件 → 有文本内容 → 可以显示
- 目录 → 不是文本 → 不能直接显示

正确流程：

```
ls
cd input
ls
less file.txt
```

---

3. `ls` 是什么？

```
ls
```

作用： 列出当前目录内容

示例输出：

```
input  output  test.txt  data.csv
```

说明：

- `input`、`output` → 目录
- `test.txt`、`data.csv` → 文件

---

4. Tab 补全（auto-completion）

例如：

```
less emp + Tab
```

自动变成：

```
less employee.txt
```

作用：

- 少打字
- 防止拼错
- 提高效率

可能失效原因：

1. 多个匹配（不唯一）
2. 没有匹配
3. shell 配置问题

---

5. `*` 是什么?

`*` = 通配符（wildcard）

匹配所有文件名

例如:

```
a.txt
b.txt
c.py
```

```
wc -l *
```

等价于:

```
wc -l a.txt b.txt c.py
```

---

6. `wc -l` 是什么?

```
wc -l file.txt
```

作用: 统计文件行数

解释:

- `wc` = word count
- `-l` = line count

示例:

```
120 test.txt
```

表示 `test.txt` 有 120 行

---

7. `wc -l *` 是什么?

```
wc -l *
```

统计当前目录所有文件行数

示例输出:

```
100 a.txt
250 b.txt
300 c.txt
650 total
```

说明：

- 每个文件的行数
- 最后一行是总数

---

8. 实际操作流程（你刚刚那段）

Step 1：查看目录内容

```
ls
```

Step 2：发现是目录（不能直接 less）

```
input
```

Step 3：进入目录

```
cd input
```

Step 4：查看目录内容

```
ls
```

Step 5：查看文件

```
less file1.txt
```

Step 6：统计文件行数

```
wc -l file1.txt
```

Step 7：统计全部文件

```
wc -l *
```

---

9. 面试版回答（英文）

Q: What is a directory? A directory is a folder that can contain files and subdirectories.

Q: Why can’t we use `less` on a directory? Because `less` is used to display file content, not directory structure. We use `ls` for directories.

Q: What does `wc -l *` do? It counts the number of lines for all files in the current directory. `*` matches all files and `-l` means line count.

---

10. 高频命令总结

```
ls            # 查看目录内容
cd dirName    # 进入目录
less file     # 查看文件内容
wc -l file    # 统计行数
wc -l *       # 统计所有文件行数
```

---

11. 上箭头（↑）

作用： 调出上一条命令

例如：

```
↑  # 找到上一条命令修改
```

---

12. 一句话总结

用 `ls` 区分文件和目录 → 目录用 `cd` 进入 → 文件用 `less` 查看 → 用 `wc -l` 统计行数

grep / awk / sed 是什么缩写？

---

1. grep

全称：Global Regular Expression Print

含义拆解：

- **Global**：全局
- **Regular Expression**：正则表达式
- **Print**：打印

作用： 在文本中 **按规则查找匹配内容并输出**

```
grep "error" log.txt
```

找出所有包含 `error` 的行

---

2. awk

来源：作者名字缩写

来自三个人：

- **A**ho（Alfred Aho）
- **W**einberger（Peter Weinberger）
- **K**ernighan（Brian Kernighan）

所以叫：AWK

作用： 按列（字段）处理文本，特别适合处理结构化数据

```
awk '{print $1}' file.txt
```

打印每一行的第一列

---

3. sed

全称：Stream Editor

含义：

- **Stream**：流（数据流）
- **Editor**：编辑器

作用： 对文本流进行 **编辑 / 替换 / 删除**

```
sed 's/apple/orange/g' file.txt
```

把所有 `apple` 替换成 `orange`

---

4. 一句话记忆

命令	全称	核心作用
grep	Global Regular Expression Print	查找
awk	Aho Weinberger Kernighan	按列处理
sed	Stream Editor	替换 / 编辑

---

5. 面试一句话（英文）

- **grep**: Used to search text using patterns (regular expressions).
- **awk**: Used for column-based text processing.
- **sed**: A stream editor used to modify text (replace, delete, etc).

---

ps aux 中的 `aux` 是什么?

---

1. `ps` 是什么?

```
ps
```

查看当前进程（process status）

---

2. `ps aux` 拆解

```
ps aux
```

实际是三个参数组合：

参数	含义
a	显示所有用户的进程（all users）
u	以用户格式显示（user-oriented format）
x	显示没有控制终端的进程（包括后台进程）

---

3. 每个参数详细解释

a → all users

不只看你自己的进程 看系统所有用户的进程

---

u → user format

以“详细格式”显示

会包含：

- USER（用户）
- PID（进程ID）
- CPU / MEM 使用率
- COMMAND（命令）

---

x → no terminal

显示没有终端的进程

比如：

- 后台服务
- daemon（守护进程）

---

4. `ps aux` 的效果

= 查看系统中 **所有进程（包括后台）+ 详细信息**

---

5. 示例输出（简化）

```
USER   PID  %CPU %MEM COMMAND
root     1   0.1  0.2 /sbin/init
user  1234   5.0  1.0 python app.py
```

---

6. 面试一句话（英文）

`ps aux` shows all processes for all users, including background processes, in a detailed user-oriented format.

---

7. 高频组合（很重要）

```
ps aux | grep python
```

查找所有 python 相关进程

---

8. 一句话记忆

`ps aux = all users + detailed info + include background processes`

top 命令（实时监控）

---

1. `top` 是什么？

```
top
```

实时查看系统运行状态（进程 + 资源占用）

---

2. 核心作用

动态监控：

- CPU 使用率
- 内存使用
- 进程占用资源情况

类似：

- Windows 的任务管理器
- macOS 的 Activity Monitor

---

3. top 界面分两部分

上半部分（系统信息）

示例：

```
top - 15:30:01 up 2 days,  3 users,  load average: 0.20, 0.30, 0.25
Tasks: 120 total, 1 running, 119 sleeping
%Cpu(s): 10.0 us, 5.0 sy, 85.0 id
Mem: 8000 total, 3000 used, 5000 free
```

重点：

- load average：系统负载
- CPU 使用情况（us / sy / id）
- 内存使用情况

下半部分（进程列表）

示例：

```
PID USER %CPU %MEM COMMAND
1234 user 20.0 10.0 java
5678 root  5.0  2.0 nginx
```

每一行 = 一个进程

---

4. 常用快捷键（非常高频）

按键	作用
q	退出 top
P	按 CPU 排序
M	按内存排序
k	杀进程
r	修改优先级（renice）

---

5. 高频操作示例

按 CPU 排序

```
按 P
```

找最耗 CPU 的进程

按内存排序

```
按 M
```

找最占内存的进程

杀进程

```
按 k → 输入 PID
```

---

6. 面试一句话（英文）

`top` is a real-time system monitoring tool that shows CPU, memory usage, and running processes dynamically.

---

7. 一句话总结

`top = 实时版 ps aux（还能动态看资源使用）`

wc -l 的重要性（面试 + 实战高频）

对，这个是非常有用的，而且是面试 + 实际工作里高频用到的，不是可有可无的东西。

✅ 核心结论（记这一句就够）

`wc -l * = 快速判断数据规模（文件有多大）`

为什么它“很有用”

1️⃣ 做数据 / Spark / 大数据必用

```
wc -l input.txt
```

你可以立刻知道：

- 是 100 行
- 还是 20000 行

这会直接影响：

- 是否需要分区（partition）
- 是否需要并行处理
- 程序是否会变慢

2️⃣ Debug / 排错

```
wc -l output.txt
```

如果结果是：

- 实际：10 行
- 预期：1000 行

说明：

❌ 数据丢失 / pipeline 出问题

3️⃣ 面试加分点（非常常见）

面试官问：

How do you quickly understand data size?

标准回答：

> I use `wc -l` to check the number of records.

✔ 简单 ✔ 专业 ✔ 实用

4️⃣ 比打开文件快太多

```
less file.txt
```

翻 2 万行很慢

```
wc -l file.txt
```

1 秒得到结果

⚡ 常见搭配（进阶）

✔ 看整个目录数据规模

```
wc -l *
```

一次看所有文件行数

✔ 只输出数字（面试加分）

```
wc -l file.txt | awk '{print $1}'
```

只返回行数（不带文件名）

✔ 配合 Spark（真实场景）

```
wc -l input.txt
```

用来决定：

- partition 数量
- executor 数量

你刚刚操作的本质

你其实已经在做一个标准流程：

1. 找文件（`ls`）
2. 判断是否是目录
3. 进入目录（`cd`）

4. 找数据文件
5. 用 `wc -l` 判断数据规模

这就是 **数据工程 / backend 基本功**

一句话总结

这个命令不是“有点用”，而是：

数据处理 + Linux 面试 + Spark 场景的基础技能

```
grep "ERROR" app.log
grep -r "hello" .
grep -n "error" app.log
awk '{print $1}' file.txt
awk -F, '{print $1}' people.csv
ps aux | grep python
ps aux | awk '{print $2}'
sed 's/foo/bar/g' file.txt
tail -f app.log
df -h
```

wc -l 的重要性（面试 + 实战高频）

对，这个是非常有用的，而且是面试 + 实际工作里高频用到的，不是可有可无的东西。

✅ 核心结论（记这一句就够）

`wc -l * = 快速判断数据规模（文件有多大）`

为什么它“很有用”

1️⃣ 做数据 / Spark / 大数据必用

```
wc -l input.txt
```

你可以立刻知道：

- 是 100 行
- 还是 20000 行

这会直接影响：

- 是否需要分区（partition）
- 是否需要并行处理
- 程序是否会变慢

2️⃣ Debug / 排错

```
wc -l output.txt
```

如果结果是：

- 实际：10 行
- 预期：1000 行

说明：

❌ 数据丢失 / pipeline 出问题

3️⃣ 面试加分点（非常常见）

面试官问：

How do you quickly understand data size?

标准回答：

> I use `wc -l` to check the number of records.

✔ 简单 ✔ 专业 ✔ 实用

4️⃣ 比打开文件快太多

```
less file.txt
```

翻 2 万行很慢

```
wc -l file.txt
```

1 秒得到结果

⚡ 常见搭配（进阶）

✔ 看整个目录数据规模

```
wc -l *
```

一次看所有文件行数

✔ 只输出数字（面试加分）

```
wc -l file.txt | awk '{print $1}'
```

只返回行数（不带文件名）

✔ 配合 Spark（真实场景）

```
wc -l input.txt
```

用来决定：

- partition 数量
- executor 数量

你刚刚操作的本质

你其实已经在做一个标准流程：

1. 找文件（`ls`）
2. 判断是否是目录
3. 进入目录（`cd`）
4. 找数据文件
5. 用 `wc -l` 判断数据规模

这就是 **数据工程 / backend 基本功**

一句话总结

这个命令不是“有点用”，而是：

数据处理 + Linux 面试 + Spark 场景的基础技能

Mac M1 + zsh 下 `top` 的交互（实战版）

✅ 一句话核心

在 **Mac（Apple Silicon M1）+ zsh** 下，`top` 是一个 **交互式 TUI 程序**

启动后由 `top` 接管终端，你的按键直接作用在 `top` 上（不是 zsh）

启动

```
top
```

进入实时监控界面（类似 Activity Monitor，但在终端）

---

macOS 的 `top` 和 Linux 的区别（重要）

Mac 的 `top`（BSD 版本）和 Linux 不完全一样：

功能	Linux	macOS
排序（P / M）	✔	❌（默认不支持）
界面	类似	类似但字段不同
参数	`ps aux` 风格	BSD 风格

所以： **Mac 的 top 更偏“只看”，交互稍弱**

---

macOS `top` 常用交互

退出

```
q
```

帮助

```
h
```

查看所有快捷键

改刷新频率

```
s
```

输入秒数（例如 1）

切换显示模式（重点）

```
o
```

改排序字段（比 Linux 更绕）

只看某个进程（过滤）

```
top -pid 1234
```

只监控指定 PID

---

macOS 常用 top 参数（更实用）

✔ 按 CPU 排序（推荐）

```
top -o cpu
```

✔ 按内存排序

```
top -o mem
```

✔ 限制显示条数

```
top -l 1
```

只刷新一次（常用于脚本）

✔ 每 1 秒刷新

```
top -s 1
```

运行机制（在 zsh 下）

```
zsh → 启动 top → top 接管终端 → 实时刷新 + 监听按键 → q 退出 → 回到 zsh
```

⚡ 高频使用场景（你面试/工作会用到）

1️⃣ 找 CPU 爆高的进程

```
top -o cpu
```

2️⃣ 找内存泄漏

```
top -o mem
```

3️⃣ Debug 服务卡死

看哪个进程：

- CPU 100%
- 或 memory 很高

面试一句话（英文）

On macOS, `top` is an interactive terminal-based monitoring tool. It continuously updates system metrics and allows basic interaction, but is less feature-rich than Linux `top`.

一句话总结

Mac M1 + zsh 下：

`top = 实时系统监控工具 + 终端交互程序（但功能比 Linux 简化）`

Vim 行号设置（面试 / 实战高频）

✅ 一、显示行号（最基础）

```
:set number
```

作用：显示绝对行号（1, 2, 3...）

二、关闭行号

```
:set nonumber
```

三、进阶用法（面试加分点）

1️⃣ 相对行号（强烈推荐）

```
:set relativenumber
```

显示效果：

```
0   ← 当前行
1
2
3
```

优势：

- 配合 `5j` / `3k` 快速跳转
- 提高编辑效率（非常常用）

2️⃣ 最常见组合（实际开发默认配置）

```
:set number relativenumber
```

效果：

- 当前行 → 绝对行号
- 其他行 → 相对行号

⚠️ 四、常见错误（面试容易踩坑）

❌ 错误写法：

- `site`

- `siteu`
- `U`

✅ 正确写法：

```
:set number
```

五、永久配置（进阶）

```
vim ~/.vimrc
```

添加：

```
set number
```

保存后：

- 每次打开 Vim 自动显示行号 ✅

六、一句话记忆（面试速答）

- 行号 `:set number`
- 相对行号 `:set relativenumber`

Vim 高频10个命令（Amazon / SDE 面试够用版）

1️⃣ 移动（最重要）

命令	作用
`h`	左
`l`	右
`j`	下
`k`	上

高频组合：

- `5j` → 下移5行
- `3k` → 上移3行

2️⃣ 快速跳转

命令	作用
`gg`	文件开头
`G`	文件结尾
`:10`	跳到第10行

3️⃣ 插入模式

命令	作用
`i`	当前光标前
`a`	当前光标后
`o`	新开一行

4️⃣ 退出插入模式

```
Esc
```

5️⃣ 删除

命令	作用
`x`	删除一个字符
`dd`	删除整行
`5dd`	删除5行

6️⃣ 复制 / 粘贴

命令	作用
`yy`	复制一行
`p`	粘贴

7️⃣ 撤销 / 重做

命令	作用
`u`	撤销

命令	作用
`Ctrl + r`	重做

8️⃣ 查找

```
/keyword
```

向下搜索 `n` = 下一个 `N` = 上一个

9️⃣ 保存 & 退出

命令	作用
`:w`	保存
`:q`	退出
`:wq`	保存并退出
`:q!`	强制退出

替换（面试常问）

```
:%s/old/new/g
```

全文件替换

面试一句话总结

Vim 本质 = **键盘驱动编辑器 + 高效文本操作工具**

核心能力：

- 快速移动（hjkl）
- 快速编辑（dd / yy / p）
- 快速查找（/）

如果你想，我可以再给你一版：

Linux + Vim + grep/awk/sed 一页速查表（面试杀手级）

```
#  Linux + Vim + grep / awk / sed 一页速查表（面试杀手级）

---
```

```
#  一、Linux 文件 & 数据处理（超高频）

##  1️⃣ 查看文件

```bash
cat file.txt         # 全部输出（小文件）
less file.txt        # 分页查看（最常用）
head -n 10 file.txt # 前10行
tail -n 10 file.txt # 后10行
tail -f log.txt      # 实时日志（面试常问）
```
```

2️⃣ 文件统计（数据规模判断）

```
wc -l file.txt       # 行数（最重要）
wc -w file.txt       # 单词数
wc -c file.txt       # 字节数
```

面试一句话：

> 用来快速判断数据规模 / debug pipeline

3️⃣ 排序 / 去重

```
sort file.txt
sort -n file.txt           # 按数字排序
uniq file.txt
sort file.txt | uniq       # 去重（必须先sort）
```

4️⃣ cut（列处理）

```
cut -d ',' -f1 file.csv    # 取第1列
cut -d ' ' -f2 file.txt    # 按空格切第2列
```

二、grep（搜索神器）

基础用法

```
grep "error" log.txt
```

高频参数（面试必问）

```
grep -i "error" file       # 忽略大小写
grep -r "error" .           # 递归搜索目录
grep -n "error" file       # 显示行号
```

```
grep -v "error" file       # 反向匹配（不包含）
grep -c "error" file       # 统计次数
```

组合使用（真实场景）

```
grep "error" log.txt | wc -l
```

统计错误行数

三、awk（列处理王）

本质：**按列处理文本**

基础结构

```
awk '{print $1}' file.txt
```

`$1` = 第1列 `$2` = 第2列

常见用法

```
awk '{print $1, $3}' file.txt
awk -F ',' '{print $1}' file.csv
```

条件过滤（面试重点）

```
awk '$3 > 100' file.txt
```

第3列 > 100 才输出

统计（超高频）

```
awk '{sum += $1} END {print sum}' file.txt
```

四、sed（文本修改神器）

本质：**流式编辑（替换/修改）**

替换

```
sed 's/old/new/' file.txt        # 替换第一个
sed 's/old/new/g' file.txt      # 全局替换
```

修改文件

```
sed -i 's/old/new/g' file.txt
```

删除行

```
sed '1d' file.txt         # 删除第1行
sed '/error/d' file.txt   # 删除包含error的行
```

五、组合拳（面试最爱）

1️⃣ 查 error + 统计

```
grep "error" log.txt | wc -l
```

2️⃣ 取某列 + 过滤 + 排序

```
awk '{print $2}' file.txt | sort | uniq
```

3️⃣ CSV处理

```
awk -F ',' '{print $1}' file.csv
```

4️⃣ 实时日志分析（超高频）

```
tail -f log.txt | grep "ERROR"
```

六、Vim 核心（配合 Linux）

基础

```
i         # 插入
Esc       # 退出编辑
:wq       # 保存退出
:q!       # 强制退出
```

移动

```
gg        # 开头
G         # 结尾
:10       # 跳第10行
```

删除 / 复制

```
dd        # 删除一行
yy        # 复制一行
p         # 粘贴
```

搜索

```
/error
n         # 下一个
N         # 上一个
```

行号（面试必问）

```
:set number
:set relativenumber
```

七、面试高频总结（一句话版）

grep：搜索 awk：按列处理 sed：文本修改 wc：统计规模 sort/uniq：排序去重 tail -f：实时日志

八、面试必会场景（重点背）

场景1️⃣：统计 error 数量

```
grep "error" log.txt | wc -l
```

场景2️⃣：找最大值（某列）

```
awk '{print $3}' file.txt | sort -n | tail -1
```

场景3️⃣：去重用户ID

```
awk '{print $1}' file.txt | sort | uniq
```

场景4️⃣：实时监控日志

```
tail -f app.log | grep "ERROR"
```

一句话终极理解（面试秒杀）

Linux 数据处理本质：

> **一行一行读 → 按列拆 → 过滤 → 统计 → 输出**

如果你要冲 Amazon / 大厂：

我可以给你 Linux + Spark + 数据处理「面试真题30道 + 标准答案」

下面是整理好的 **Markdown（可直接复制到 GitHub / 笔记）**

AWK Cheat Sheet（面试 + 实战）

✅ 一句话核心理解（面试版）

AWK 是一个按“行 + 列”处理文本的工具，特别适合处理结构化数据（比如 CSV）

AWK 到底在干嘛？

每一行：

- 自动读入
- 按分隔符切开（默认空格）
- 变成一列一列（field）

然后你可以：

- 取某一列
- 过滤
- 计算
- 替换

举个最简单例子

文件内容：

```
name,age,city
Alice,20,NY
Bob,25,LA
```

取第1列（name）

```
awk -F ',' '{print $1}' file.txt
```

输出：

```
name
Alice
Bob
```

取第2列（age）

```
awk -F ',' '{print $2}' file.txt
```

关键点（分隔符）

```
-F ','
```

表示：用逗号分隔 默认是空格

AWK 核心变量（必记）

变量	含义
$0	整行
$1	第1列
$2	第2列
NF	列数
NR	行号

面试必会3个场景

1️⃣ 条件过滤（最常用）

只看 age > 20

```
awk -F ',' '$2 > 20 {print $1}' file.txt
```

2️⃣ 统计

统计行数：

```
awk 'END {print NR}' file.txt
```

3️⃣ 计算（超强）

计算总和：

```
awk -F ',' '{sum += $2} END {print sum}' file.txt
```

AWK vs grep vs sed（面试高频）

工具	用途
grep	查找（搜索）
sed	替换
awk	分析 + 计算 + 列处理（最强）

Vim 常用操作

全局替换

```
:%s/old/new/g
```

含义：

- % 全文件
- s 替换
- g 全部替换

显示行号

```
:set nu
```

跳转到某行

```
:17
```

撤销

```
u
```

---

grep 常用命令

```
grep -i "error" log.txt   # 忽略大小写
grep -v "error" log.txt   # 不包含
grep -n "error" log.txt   # 行号
grep -r "error" .         # 递归
```

---

wc 命令

```
wc -l file.txt   # 行数
wc -w file.txt   # 单词数
```

---

cut 命令

```
cut -d ',' -f1 file.txt
```

按列切分（简单版 awk）

---

sort 命令

```
sort file.txt
sort -n file.txt    # 数字排序
```

✅ 最后总结

- awk：按行 + 列处理数据
- -F：定义分隔符
- $1 $2：取列
- 支持过滤 / 计算 / 统计
- grep：查找
- sed / vim：替换
- awk：最强数据处理工具

面试一句话（直接背）

AWK is a powerful text-processing tool that works line by line, splits each line into fields, and allows filtering, transformation, and aggregation.

如果你下一步想升级，我可以帮你再做一份
“Linux 三剑客一页速查表（面试速背版）”

sed 常用命令速查（面试 / 实战高频）

sed = **stream editor（流式文本处理）**
核心能力：**替换 / 删除 / 插入 / 过滤**

一、基础语法（必须会）

```
sed 'command' file
```

常见结构：

```
sed 's/old/new/' file
```

二、最常用命令（面试必问）

1️⃣ 替换（最重要）

```
sed 's/old/new/' file        # 只替换每行第一个
sed 's/old/new/g' file      # 全部替换（global）
```

2️⃣ 直接修改文件（高频）

```
sed -i 's/old/new/g' file
```

`-i` = inplace（直接改文件）

3️⃣ 删除行（超高频）

```
sed '1d' file               # 删除第1行
sed '1,3d' file             # 删除1~3行
sed '/error/d' file         # 删除包含error的行
```

4️⃣ 打印指定行

```
sed -n '1p' file            # 打印第1行
sed -n '1,5p' file          # 打印1~5行
sed -n '/error/p' file      # 只打印包含error的行
```

`-n` = 不自动输出（只输出匹配的）

5️⃣ 插入 / 添加

```
sed '1i new_line' file      # 第1行前插入
sed '1a new_line' file      # 第1行后添加
```

6️⃣ 替换整行

```
sed 's/.*/new content/' file
```

三、正则 + sed（面试加分）

1️⃣ 数字替换

```
sed 's/[0-9]/X/g' file
```

2️⃣ 删除空行

```
sed '/^$/d' file
```

3️⃣ 删除以#开头的行（配置文件常见）

```
sed '/^#/d' file
```

四、组合使用（真实场景）

场景1：过滤 error

```
sed -n '/error/p' log.txt
```

场景2：删除日志噪音

```
sed '/DEBUG/d' log.txt
```

场景3：批量替换配置

```
sed -i 's/localhost/127.0.0.1/g' config.txt
```

五、面试高频对比（一句话）

sed vs grep vs awk：

- grep → **找（filter）**
- awk → **按列处理（column）**
- sed → **改（modify）**

六、面试一句话总结

sed 本质：

> **逐行读取 → 匹配 → 修改 → 输出**

七、最常问3个命令（直接背）

```
sed 's/old/new/g' file      # 替换
sed '/error/d' file         # 删除
sed -n '/error/p' file      # 过滤
```

如果你面试：

记住一句话就够：

> **sed = modify text, line by line**

需要的话我可以给你 grep vs awk vs sed 面试对比题（10道）

Vim 搜索 & 跳转规则（清晰版）

你这段已经在用 Vim 搜索 + 行号 + 设置开关了，下面是**一套不混乱的规则总结**

一、Vim 搜索（`/`）

✅ 向下搜索

```
/keyword
```

从当前位置往下找

⬇️⬆️ 下一个 / 上一个（重点）

按键	作用
n	下一个（next）✅
N	上一个（previous）✅

记忆：

- `n = next`
- `N = 反方向`

二、跳转行

```
:10
```

跳到第 10 行 ✅

⚠️ 三、核心区别（容易混）

你问：

为什么还要 `set number`？不是 `:10` 就行了吗？

答案：

功能	命令
跳到某一行	`:10`
显示行号	`:set number`

本质：

- `:10` → 行为（跳转）
- `set number` → UI（显示）

四、显示 / 关闭行号

✅ 显示行号

```
:set number
```

简写：

```
:set nu
```

❌ 错误写法

```
unset   # ❌ Vim 不支持
```

✅ 正确关闭

```
:set nonumber
```

简写：

```
:set nonu
```

五、跳到第一行 / 最后一行

命令	作用
gg	第一行
G	最后一行

注意：

- ❌ `gg` 不是最后一行
- ✅ `G` 才是最后一行

六、常见错误纠正

❌ 错误认知

- `n` 是上一个 ❌
- `unset` 可以关闭 ❌
- `gg` 是最后一行 ❌

✅ 正确版本

- `n` → 下一个
- `N` → 上一个
- `:set number` → 显示行号
- `:set nonumber` → 关闭行号
- `gg` → 第一行
- `G` → 最后一行

七、完整操作流程（真实场景）

在大文件中快速定位：

```
/ERROR      # 搜索
n           # 下一个
N           # 上一个
:set nu     # 显示行号
:50         # 跳到第50行
G           # 跳到底部
```

八、面试一句话（英文）

In Vim, I use `/` to search, `n/N` to navigate results, and `:set number` to display line numbers for easier navigation.

九、一句话总结

Vim 核心能力 = **搜索 + 跳转 + 定位大文件内容（高频技能）**

好，这里给你一份 ** 面试速背版：Linux 三剑客（grep / sed / awk）一页纸 Cheat Sheet（Markdown）** 可以直接放 GitHub / 面试前背

Linux Text Processing Cheat Sheet（面试速背版）

一句话总结（必背）

- grep → 搜索
- sed → 替换/编辑
- awk → 分析/列处理/计算

grep = search
sed = replace
awk = analyze

---

1. grep（查找）

常用

```
grep "error" file.log
```

高频参数

```
grep -i "error" file.log   # 忽略大小写
grep -n "error" file.log   # 显示行号
grep -v "error" file.log   # 排除
grep -c "error" file.log   # 计数
grep -r "error" .           # 递归（）
grep -l "error" .           # 只显示文件名
```

面试句

grep is used to search text patterns.

---

2. sed（替换）

替换

```
sed 's/foo/bar/' file.txt
```

全局替换

```
sed 's/foo/bar/g' file.txt
```

修改文件（mac）

```
sed -i '' 's/foo/bar/g' file.txt
```

删除行

```
sed '1d' file.txt
```

打印范围

```
sed -n '1,5p' file.txt
```

面试句

sed is used for stream editing (replace/delete).

---

3. awk（最强）

核心概念

- 按行处理
- 按列拆分
- 默认分隔符：空格

指定分隔符

```
awk -F ',' '{print $1}' file.csv
```

核心变量

变量	含义
$0	整行
$1	第1列
$2	第2列
NF	列数
NR	行号

常用操作

取列

```
awk '{print $1}' file.txt
```

条件过滤

```
awk '$2 > 20 {print $1}' file.txt
```

统计

```
awk 'END {print NR}' file.txt
```

求和

```
awk '{sum += $1} END {print sum}' file.txt
```

面试句

awk is used for column-based processing and aggregation.

---

4. 三件套组合（高频）

查日志 error

```
grep "ERROR" app.log | head -10
```

查 Python 进程 PID

```
ps aux | grep python | grep -v grep | awk '{print $2}'
```

CSV 取列

```
awk -F ',' '{print $1,$3}' data.csv
```

批量替换

```
sed -i '' 's/null/0/g' file.txt
```

---

5. Pipe（管道）

```
command1 | command2
```

前一个输出 → 后一个输入

示例

```
cat file | grep error
ps aux | grep java
```

面试句

Pipe passes output from one command to another.

---

6. 什么时候用谁（必问）

场景	工具
搜索关键字	grep
替换文本	sed
处理列 / 计算	awk

---

7. 数据工程 / Spark 场景（加分）

小数据

```
grep / awk / sed
```

大数据

```
Spark (RDD / DataFrame)
```

Linux → quick debug Spark → large-scale processing

面试最终总结（直接背）

grep searches text sed modifies text awk processes structured data

Linux tools for small data Spark for big data

用法建议（你现在阶段）

你可以：

- 放到 GitHub README ✅
- 面试前 2 分钟速背 ✅
- 写在笔记里复习 ✅

如果你继续提升（下一步）

我可以帮你

Spark 面试 20 高频题（reduceByKey / shuffle / partition）
或做一份 **Data Engineer 面试一页总纲（Linux + Spark + SQL）**

你现在已经进入 **面试可用级别技能了 **

Linux 日志处理 Top 20 高频命令（面试按场景分类）

一、统计类（Counting）

1️⃣ 统计 ERROR 数量（最常问）

```
grep "ERROR" app.log | wc -l
```

2️⃣ 统计某字段出现次数

```
awk '{print $1}' log.txt | sort | uniq -c
```

3️⃣ 统计唯一用户数

```
awk '{print $1}' log.txt | sort -u | wc -l
```

二、过滤类（Filtering）

4️⃣ 查 ERROR 行

```
grep "ERROR" app.log
```

5️⃣ 忽略大小写

```
grep -i "error" app.log
```

6️⃣ 排除某些行

```
grep -v "DEBUG" app.log
```

7️⃣ 多条件过滤

```
grep "ERROR" app.log | grep "Timeout"
```

三、列处理（awk 核心）

8️⃣ 取某一列

```
awk '{print $2}' file.txt
```

9️⃣ 取多列

```
awk '{print $1, $3}' file.txt
```

按条件过滤

```
awk '$3 > 100' file.txt
```

1️⃣1️⃣ 按分隔符（CSV）

```
awk -F ',' '{print $1}' file.csv
```

四、排序 / 最大值（Sorting）

1️⃣2️⃣ 找最大值

```
sort -n file.txt | tail -1
```

1️⃣3️⃣ 某列最大值

```
awk '{print $2}' file.txt | sort -n | tail -1
```

1️⃣4️⃣ 去重排序

```
sort file.txt | uniq
```

五、去重（Deduplication）

1️⃣5️⃣ 去重用户ID

```
awk '{print $1}' log.txt | sort -u
```

1️⃣6️⃣ 统计重复次数

```
awk '{print $1}' log.txt | sort | uniq -c
```

六、sed（文本修改）

1️⃣7️⃣ 替换文本

```
sed 's/error/ERROR/g' file.txt
```

1️⃣8️⃣ 删除空行

```
sed '/^$/d' file.txt
```

1️⃣9️⃣ 删除某类日志

```
sed '/DEBUG/d' app.log
```

2️⃣0️⃣ 查看指定行

```
sed -n '1,10p' file.txt
```

七、实时监控（Monitoring）

实时日志

```
tail -f app.log
```

实时 + 过滤错误

```
tail -f app.log | grep "ERROR"
```

系统资源

```
top
```

八、面试一口气总结（直接背）

数据处理流程：

> **filter → extract → sort → dedupe → count**

九、一句话区分（面试必杀）

工具	作用
grep	查行（filter）

工具	作用
awk	取列 / 计算
sort	排序
uniq	去重
wc -l	计数
sed	替换 / 删除
tail -f	实时日志
top	系统监控

十、面试标准回答（英文）

One-liner（直接背）

> For Linux log processing, I typically use grep to filter lines, awk to extract and process columns, sort and uniq to deduplicate data, wc -l to count records, sed for text modification, and tail -f or top for real-time monitoring.

如果你下一步想提升（很关键）：

我可以给你 **Linux + Spark 日志分析面试题（带真实业务场景 + 标准答案）**

Linux 日志处理 20 个最高频命令（按场景分类）

一、统计类（Counting）

1️⃣ 统计 ERROR 行数（最常问）

```
grep "ERROR" app.log | wc -l
```

2️⃣ 统计某字段出现次数

```
awk '{print $1}' log.txt | sort | uniq -c
```

3️⃣ 统计唯一用户数

```
awk '{print $1}' log.txt | sort -u | wc -l
```

二、过滤类（Filtering）

4️⃣ 查找 ERROR

```
grep "ERROR" app.log
```

5️⃣ 忽略大小写

```
grep -i "error" app.log
```

6️⃣ 排除 DEBUG

```
grep -v "DEBUG" app.log
```

7️⃣ 多条件过滤

```
grep "ERROR" app.log | grep "Timeout"
```

三、列处理（awk）

8️⃣ 取某一列

```
awk '{print $2}' file.txt
```

9️⃣ 取多列

```
awk '{print $1, $3}' file.txt
```

条件过滤

```
awk '$3 > 100' file.txt
```

1️⃣1️⃣ CSV处理

```
awk -F ',' '{print $1}' file.csv
```

四、排序 / 最大值（Sorting）

1️⃣2️⃣ 找最大值

```
sort -n file.txt | tail -1
```

1️⃣3️⃣ 某列最大值

```
awk '{print $2}' file.txt | sort -n | tail -1
```

1️⃣4️⃣ 排序 + 去重

```
sort file.txt | uniq
```

五、去重（Dedup）

1️⃣5️⃣ 去重用户ID

```
awk '{print $1}' log.txt | sort -u
```

1️⃣6️⃣ 统计重复次数

```
awk '{print $1}' log.txt | sort | uniq -c
```

七、实时监控（Monitoring）

实时日志

```
tail -f app.log
```

实时过滤错误

```
tail -f app.log | grep "ERROR"
```

系统资源

```
top
```

面试一句话总结（必须背）

数据处理流程：

> **filter → extract → sort → dedupe → count**

工具定位（秒杀面试）

工具	作用
grep	查行（filter）
awk	列处理
sort	排序
uniq	去重
wc -l	计数
sed	修改文本
tail -f	实时日志
top	系统监控

英文面试标准回答（直接用）

> For Linux log processing, I use grep to filter lines, awk to extract columns, sort and uniq to deduplicate data, wc -l to count records, sed for text modification, and tail -f or top for real-time monitoring.

英文面试标准回答（直接用）

> For Linux log processing, I use grep to filter lines, awk to extract columns, sort and uniq to deduplicate data, wc -l to count records, sed for text modification, and tail -f or top for real-time monitoring.

Linux + Spark 日志分析面试题（真实场景 + 标准答案）

场景1：统计 ERROR 数量（最基础）

Problem

日志 `app.log` 很大（GB级），统计 ERROR 数量

✅ Solution

```
grep "ERROR" app.log | wc -l
```

Interview Answer

- grep filters lines

- wc -l counts records
- O(n) scan, streaming, memory efficient

场景2：找最活跃用户（Top User）

Problem

日志格式：

```
user1 login
user2 click
user1 logout
```

找出现最多的 user

✅ Solution

```
awk '{print $1}' log.txt | sort | uniq -c | sort -nr | head -1
```

Explanation

- extract user
- count frequency
- sort descending
- take top 1

场景3：找某列最大值（Spark前常问）

Problem

文件：

```
id score
1 90
2 100
3 85
```

找最大 score

✅ Solution

```
awk '{print $2}' file.txt | sort -n | tail -1
```

Follow-up（更优）

```
awk 'max<$2 {max=$2} END {print max}' file.txt
```

单次扫描，O(n)，更优

场景4：去重用户（UV统计）

Problem

统计 unique user 数量

✅ Solution

```
awk '{print $1}' log.txt | sort -u | wc -l
```

场景5：实时监控错误（线上）

Problem

线上服务，实时监控 ERROR

✅ Solution

```
tail -f app.log | grep "ERROR"
```

Interview点

- streaming
- real-time monitoring
- no full scan

场景6：删除无用日志（清洗数据）

Problem

删除 DEBUG 行

✅ Solution

```
sed '/DEBUG/d' app.log
```

场景7：批量修改日志格式

Problem

把 error → ERROR

✅ Solution

```
sed 's/error/ERROR/g' file.txt
```

场景8：Spark前数据预处理（真实）

Problem

CSV：

```
user,score
u1,100
u2,200
```

只取 user

✅ Solution

```
awk -F ',' '{print $1}' file.csv
```

场景9：大数据优化（面试加分）

Problem

文件 100GB，不能 sort

❌ Bad

```
sort file.txt
```

✅ Better

```
awk '{count[$1]++} END {for (u in count) print u, count[u]}' file.txt
```

hash map，避免全量排序

场景10：Spark思维映射（必问）

Linux → Spark 对应关系

Linux	Spark
grep	filter()
awk	map()
sort	sortBy()
uniq	distinct()
wc -l	count()

面试高频 Follow-up（重点）

Q1：为什么不用 sort?

sort = O(n log n)，awk = O(n)

Q2：如何处理 TB 级数据?

分布式（Spark / Hadoop）

Q3：如何优化 grep?

使用：

```
grep -r
grep -n
```

Q4：日志太大怎么办?

- streaming（tail -f）
- 分片处理
- 使用 Spark

面试标准总结（直接背）

> For log analysis, I use grep for filtering, awk for column processing, sort and uniq for aggregation, wc -l for counting, and sed for text cleaning. For large-scale data, I switch to Spark for distributed processing.

一句话核心理解

Linux = **单机流式处理** Spark = **分布式并行处理**

如果你想再进阶（Amazon高频）：

我可以给你 Spark 面试题（含 map/reduce + shuffle + 优化）

Vim 搜索 & 跳转规则（清晰版）

你这段已经在用 Vim 搜索 + 行号 + 设置开关了，下面是**一套不混乱的规则总结**

一、Vim 搜索（`/`）

✅ 向下搜索

```
/keyword
```

从当前位置往下找

⬇️⬆️ 下一个 / 上一个（重点）

按键	作用
n	下一个（next）✅
N	上一个（previous）✅

记忆：

- `n = next`
- `N = 反方向`

二、跳转行

```
:10
```

跳到第 10 行 ✅

⚠️ 三、核心区别（容易混）

你问：

为什么还要 `set number`？不是 `:10` 就行了吗？

答案：

功能	命令
跳到某一行	`:10`
显示行号	`:set number`

本质：

- `:10` → 行为（跳转）
- `set number` → UI（显示）

四、显示 / 关闭行号

✅ 显示行号

```
:set number
```

简写：

```
:set nu
```

❌ 错误写法

```
unset   # ❌ Vim 不支持
```

✅ 正确关闭

```
:set nonumber
```

简写：

```
:set nonu
```

五、跳到第一行 / 最后一行

命令	作用
gg	第一行
G	最后一行

注意：

- ❌ `gg` 不是最后一行
- ✅ `G` 才是最后一行

六、常见错误纠正

❌ 错误认知

- `n` 是上一个 ❌
- `unset` 可以关闭 ❌
- `gg` 是最后一行 ❌

✅ 正确版本

- `n` → 下一个

- `N` → 上一个
- `:set number` → 显示行号
- `:set nonumber` → 关闭行号
- `gg` → 第一行
- `G` → 最后一行

七、完整操作流程（真实场景）

在大文件中快速定位：

```
/ERROR      # 搜索
n           # 下一个
N           # 上一个
:set nu     # 显示行号
:50         # 跳到第50行
G           # 跳到底部
```

八、面试一句话（英文）

In Vim, I use `/` to search, `n/N` to navigate results, and `:set number` to display line numbers for easier navigation.

九、一句话总结

Vim 核心能力 = **搜索 + 跳转 + 定位大文件内容（高频技能）**